動態規劃

遞迴把問題拆開；動態規劃進一步辨認哪些子問題其實相同，讓每個狀態只求解一次。

本章從狀態與依賴開始，依序處理 **鋼條切割 → 背包 → 最長共同子序列與序列比對 → 矩陣鏈與最佳二元搜尋樹**。

前置知識為遞迴、漸近分析、陣列與樹。重點是把問題定義、遞迴式、填表順序與答案重建連成同一個演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先決定狀態代表什麼

設計問題	必須說清楚的內容
狀態是否足夠？	相同狀態的未來選擇與成本必須相同。
遞迴是否完整？	每種合法最優解都要落入某個候選分支。
先算哪些狀態？	邊界與依賴決定合法的填表順序。
如何交回答案？	最優值、選擇紀錄與重建路徑各有用途。

每個模型都先固定合法解與成本，再比較時間、空間及是否能重建一個最優解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態規劃：先決定什麼才算同一問題

動態規劃（dynamic programming, DP）把重複出現的子問題合併為狀態，保存答案，再組合出原問題答案。

- **狀態語義**：此格究竟最佳化哪一組合法解？
- **轉移與證明**：候選能否涵蓋所有最佳解，而且都合法？
- **計算與重建**：依賴先算，並記錄達成最佳值的選擇。

最佳子結構支持正確組合；子問題重疊解釋為何快取能節省重算。兩者不能代替狀態定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態充分性：忘記歷史之前先檢查

若兩段歷史被合併為同一狀態，它們必須具有相同的後續合法選擇、後續成本規則與剩餘目標。

已累積價值可以分開記帳；狀態需保留會改變未來的資訊。

例如切鋼條：每段價格只取決於長度、切割免費、沒有段數上限。剩餘長度同為 j，先前如何切不影響未來，因此 $r[j]$ 足夠。

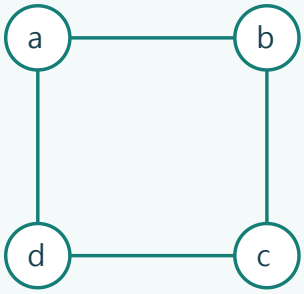
若新增「最多切成 k 段」，只記 j 便不足，還需記剩餘可用段數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最長簡單路：端點狀態會漏掉資源限制

無向圖的邊為 ab, bc, cd, da，每條邊長為1；簡單路不可重複頂點。

端點相同，不代表餘下資源相同



最佳整路 / Whole optimum: a → b → c → d

後綴 / Suffix: c → d (1)

獨立最長 / Independent: c → b → a → d (3)

拼接 / Splice: a → b → c → b → a → d

重複 a, b：非法

$a \to b \to c \to d$ 是 a 到 d 的最長簡單路，長度3。但其中 $c \to d$ 並非端點 c, d 間最長路；$c \to b \to a \to d$ 長度3。直接替換得到 a, b, c, b, a, d，已不合法。

失敗原因：端點沒有記錄前綴已使用的頂點，不能任意拼接各端點間最佳解。

擴充狀態可以修復，但可能付出指數成本

固定起點 s，令 $D[S, v]$ 為「恰好使用頂點集合 S、從 s 到 v 的最長簡單路長度」，不存在則為 $-\infty$。

$$D[\{s\}, s] = 0, \qquad D[S, v] = \max_{\substack{u \in S \setminus \{v\} \\ uv \in E}} (D[S \setminus \{v\}, u] + 1).$$

其他基底為 $-\infty$；僅計算 $s, v \in S$、$v \neq s$ 的非基底狀態。答案為 $\max_{S \ni s,t} D[S, t]$。

集合大小嚴格增加，形成狀態 DAG；至多 $O(2^{|V|}|V|^2)$ 時間。這是指數 DP，不是多項式解。若原圖本身是 DAG，端點最長路可直接按拓撲序計算。

切鋼條：問題條件與狀態

給定整數長度 $n \geq 0$，價格 $p[1..n]$。將整條鋼條切成正整數長度的段，每段都出售；切割免費，同長度可以出現多次。

$$r[j] = \text{恰好用完長度 } j \text{ 的最大總收益}, \qquad r[0] = 0.$$

本例價格（收益單位）：

長度 i	1	2	3	4
$p[i]$	1	5	8	9

求最佳值 $r[4]$，並輸出每段長度。整段出售也是合法方案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第一段選擇導出遞迴式

任何非空切法都有第一段長度 i，$1 \leq i \leq j$；第一段直接出售，只有餘段繼續切。

$$r[j] = \max_{1 \leq i \leq j} \{p[i] + r[j-i]\}, \qquad j \geq 1.$$

- $i = j$：不再切割，餘段為空，收益 $p[j] + r[0]$。
- 不把第一段寫成 $r[i]$：此處 i 表示一段成品的長度。
- 只要長度下降，就能以 j 歸納證明計算正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩向證明：不漏解，也不引入假解

令 $OPT(j)$ 為真正最佳收益，並假設所有更短狀態均已正確。

上界方向：任一最佳切法第一段為 i，其餘段收益不超過 $r[j-i]$。所以 $OPT(j) \le p[i] + r[j-i] \le \max_i(p[i] + r[j-i])$。

下界方向：對每個 i，一段長度 i 接上長度 $j-i$ 的最佳切法，長度恰為 j，全部合法。因此 $p[i] + r[j-i] \le OPT(j)$；取最大仍成立。

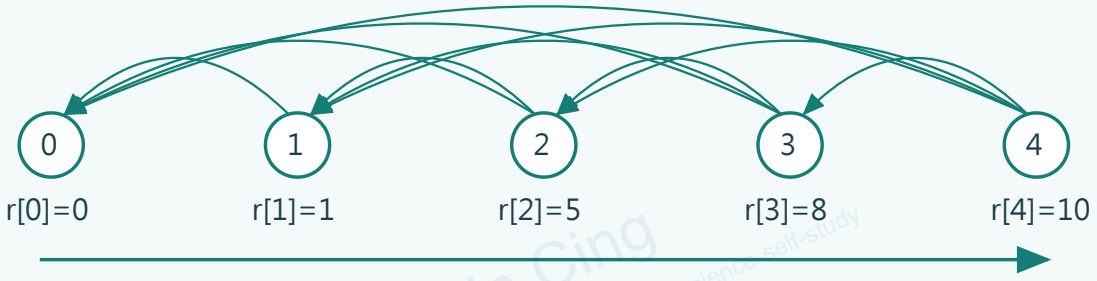
兩式合起來得到等號。替換餘段不會改變第一段合法性，是最佳子結構成立的關鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重疊子問題與依賴 DAG

箭頭 $j \to k$ 表示計算 $r[j]$ 需要 $r[k]$。

箭頭：狀態 → 所需子問題



計算次序：0 → 1 → 2 → 3 → 4

5 個狀態；10 條依賴；j 嚴格下降

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先完成較短依賴，再計算較長狀態

所有 $k < j$ 都是 $r[j]$ 的候選依賴。例如 $r[3]$、$r[4]$ 都需要 $r[2]$。合併同長度呼叫後，這份答案只需保存一次。

長度4的依賴 DAG 有5個狀態 $0, 1, 2, 3, 4$。狀態 j 有 j 條向外的依賴邊，因此總數為 $1 + 2 + 3 + 4 = 10$。

依 $0, 1, 2, 3, 4$ 填表，方向與依賴箭頭相反。計算 $r[j]$ 時，所有需要的 $r[k]$ 都已完成。依賴長度嚴格下降，因此不會形成循環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

記憶化：每個狀態只展開一次

```
memo[0] = 0; 其餘為 UNKNOWN
solve(j):
    if memo[j] is not UNKNOWN: return memo[j]
    q = -∞
    for i = 1 .. j:
        q = max(q, p[i] + solve(j-i))
    memo[j] = q
    return q
```

呼叫從目標往下展開；重複呼叫直接讀表。以獨立的 `UNKNOWN` 記未計算，不能把合法的負收益當作未知。

本題所有長度都會到達，故仍需 $\Theta(n^2)$ 時間、$\Theta(n)$ 表格與至多 $O(n)$ 呼叫堆疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由小到大填表，同時記下第一段

```
r[0] = 0
for j = 1 .. n:
    r[j] = -∞
    for i = 1 .. j:           # i 由小到大
        q = p[i] + r[j-i]
        if q > r[j]:          # 同值保留較小的第一段
            r[j] = q
            first[j] = i
```

迴圈不變量：開始算 j 時，$r[0..j-1]$ 全部正確。每個候選都只讀這些已完成的值。

時間為 $\sum_{j=1}^{n} j = n(n+1)/2 = \Theta(n^2)$；值表與選擇表共 $\Theta(n)$ 空間。

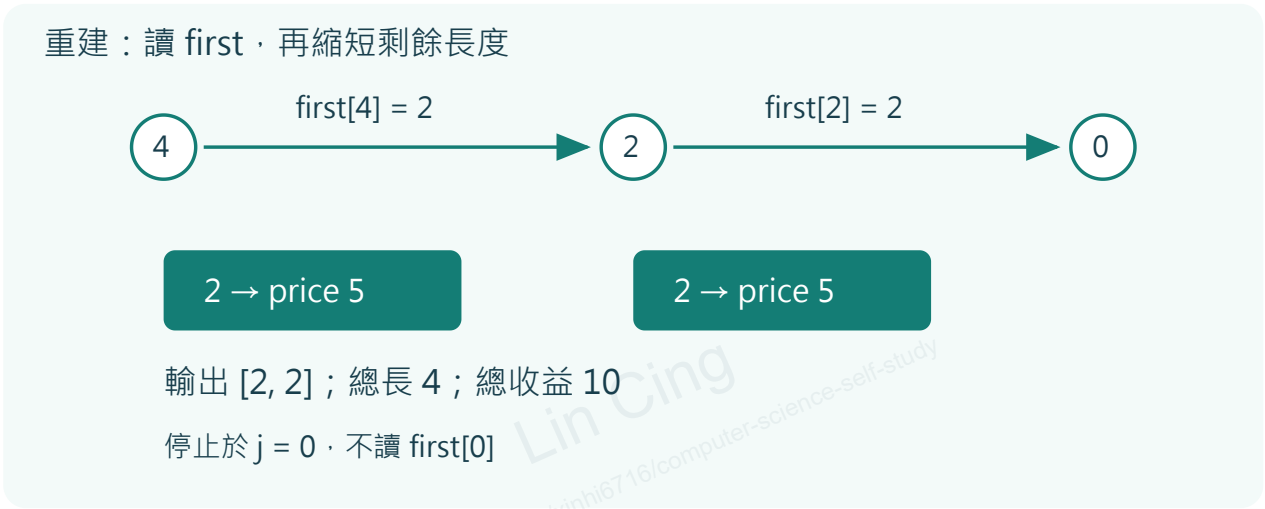
完整數例：每個候選都可追查

依 $i = 1, 2, \ldots, j$ 列出候選 $p[i] + r[j-i]$。

j	候選收益（依 i 遞增）	掃描中的最大值	$r[j]$	first[j]
0	空切法	—	0	—
1	$1+0=1$	1	1	1
2	$1+1=2,\ 5+0=5$	2, 5	5	2
3	$1+5=6,\ 5+1=6,\ 8+0=8$	6, 6, 8	8	3
4	$1+8=9,\ 5+5=10,\ 8+1=9,\ 9+0=9$	9, 10, 10, 10	10	2

收益最大的第一段不一定是售價最高的段：$p[4]=9$，而兩段長度2可得10。

沿記錄的選擇還原最佳切法



此例 $first[4]=2$、$first[2]=2$。剩餘長度依序為 $4\to2\to0$，輸出 $[2,2]$。核對長度和 $2+2=4$，收益 $5+5=10=r[4]$。

記錄的切法為何維持最佳收益

剩餘長度為 j 時，已輸出各段的長度和為 $n-j$，且其收益加上 $r[j]$ 等於 $r[n]$。初始 $j=n$、尚未輸出任何段，因此兩項性質都成立。

令 $i=first[j]$。輸出 i 並把 j 換成 $j-i$，已輸出長度恰增加 i。由 $r[j]=p[i]+r[j-i]$，已輸出收益的增加量，恰等於剩餘最佳收益的減少量，因此兩項性質都維持。

到達 $j=0$ 時，已輸出長度為 n，且 $r[0]=0$，故已輸出收益為 $r[n]$。本例輸出兩段長度2，總長度4、收益10。

重建迴圈在剩餘長度為零時停止

填表完成後，每個正長度 j 的 `first[j]` 都記錄一個最佳第一段。沿這些選擇即可輸出完整切法。

```
j = n; pieces = []
while j > 0:
    i = first[j]
    append i to pieces
    j = j - i
return pieces
```

每個選擇皆滿足 $1 \le first[j] \le j$。每次至少減1，故迴圈至多執行 n 次便在零停止，重建需 $O(n)$ 時間。此時不變量保證總長度為 n、總收益為 $r[n]$。

$n = 0$ 時不進入迴圈，直接回傳空串列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

記憶化與填表是在排同一張圖

面向	Top-down memoization	Bottom-up tabulation
啟動方式	從目標遞迴要求依賴	按合法順序逐狀態計算
實際狀態	僅展開可達狀態	通常填預先指定的範圍
額外成本	呼叫堆疊、快取查詢	通常無遞迴堆疊
本例時間／表格	$\Theta(n^2)$／$\Theta(n)$	$\Theta(n^2)$／$\Theta(n)$

記憶化不保證比填表快；填表也不必是二維矩形。兩者都必須先有正確轉移與無循環的依賴順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

邊界條件會改變答案

必須用完整條：若 $p[1] = -2, p[2] = -5$，則 $r[1] = -2$、$r[2] = \max(-4, -5) = -4$。初始最大值必須為 $-\infty$，不能為0。

允許丟棄剩餘材料：改成 $r[j] = \max(0, \max_i(p[i] + r[j-i]))$，同一負價例答案為0；重建也需能記錄「停止並丟棄」。這是另一個問題。

空輸入：$n = 0$，價格表可為空，收益0、切法空。若 $n > 0$，本章假設每個長度 $1..n$ 都有價格；缺價長度不能直接當成0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

換條件時，重新檢查轉移

固定每次實際切割費用 $c \geq 0$：最後整段出售時不收切割費。

$$r[j] = \max_{1 \leq i \leq j} \left(p[i] + r[j-i] - c\,\mathbf{1}_{i<j} \right).$$

原例 $c = 2$：兩段2收益 $5 + 5 - 2 = 8$，整段4收益9，因此最佳改為不切。

這仍只需長度狀態；若限制成品段數或價格依先前選擇改變，就應擴充狀態。判斷依據是未來是否合法、成本能否只由狀態決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用可檢查的五件事設計 DP

1. 精確寫出每個狀態的合法解集合與基底。
2. 指定第一個／最後一個決策，枚舉全部合法候選。
3. 證明最佳解落在候選中，而且候選都能組成合法解。
4. 找出嚴格下降的尺度，安排依賴順序。
5. 以「狀態數 × 每狀態候選成本」估算，並另外驗證重建。

本例：長度狀態、第一段決策、兩向證明、遞減長度、二次時間；輸出切法再核對總長度與收益。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

0/1 背包：每件只能選一次

有 n 件物品，重量 w_i 為正整數，價值 v_i，容量 W 為非負整數。

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{使} \quad \sum_{i=1}^{n} w_i x_i \le W, \quad x_i \in \{0,1\}.$$

允許空背包，所以最優值至少為 0；負價值物品不必取。物品可有相同重量或價值，但仍是不同的件。 整數容量是這個容量索引 DP 的前提；任意實數重量不能直接當陣列索引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態為何要保留物品範圍

定義 $D[i,c]$：只用物品 $1,\dots,i$、總重至多 c 的最大價值。答案為 $D[n,W]$。 只記「剩餘容量」不夠：相同容量下，可用物品不同，答案也可能不同。以物品前綴限制可用集合，才不會重複拿同一件。 對固定 (i,c)，未來選擇只取決於此前綴與容量；不同歷史若到達相同狀態，較差的價值不可能改善任何延伸。這裡假設價值可相加，沒有物品間相依限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

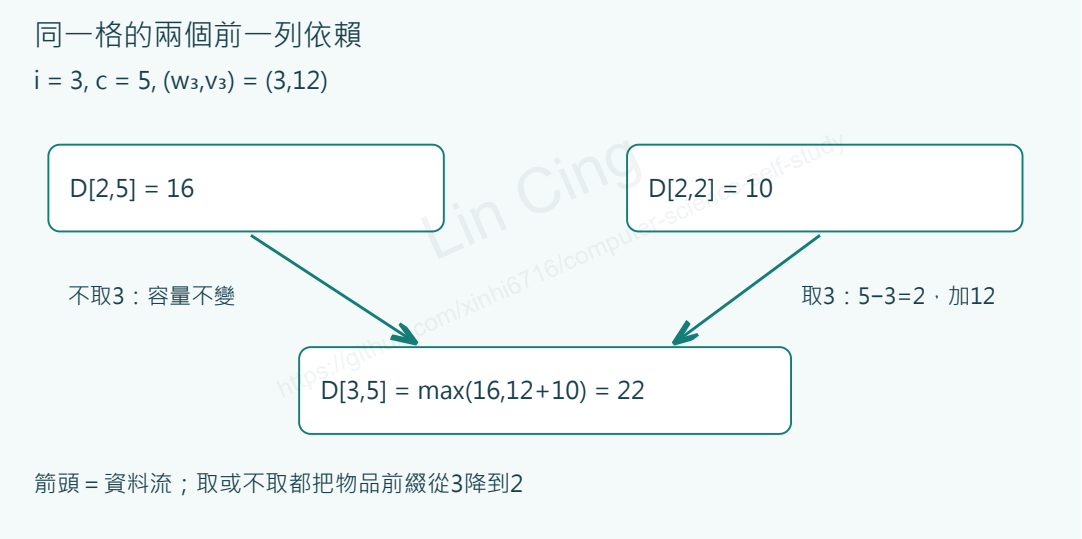
兩個互斥分支導出遞迴

$$D[0,c]=0,\qquad D[i,0]=0.$$

若 $c<w_i$，則 $D[i,c]=D[i-1,c]$；否則

$$D[i,c]=\max\{D[i-1,c],\ v_i+D[i-1,c-w_i]\}.$$

不取 i 保留容量；取 i 要扣去 w_i，且兩分支都只剩前 $i-1$ 件。



遞迴的上下界證明

任取 (i, c) 的可行集合 S。若 $i \notin S$，其價值至多 $D[i-1, c]$；若 $i \in S$，移除 i 後只用前 $i-1$ 件，重量至多 $c - w_i$，故價值至多 $v_i + D[i-1, c-w_i]$。因此最優值不超過候選最大值。 反方向：不取分支的最優集合可直接使用；取分支的最優集合不含 i，加上 i 後仍不超重，故兩個候選都是可達值。最大候選不超過最優值。 配合 $i = 0$ 的基底，對 i 歸納即得正確性。若移除 i 後不是剩餘問題的最優解，可換成更好的解並加回 i，與原解最優矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一數例：輸入與填表順序

容量 $W = 5$；物品 (w_i, v_i) 為 $(2, 10), (1, 6), (3, 12)$。

列是可用前綴，欄是容量上限，並非必須填滿的重量。

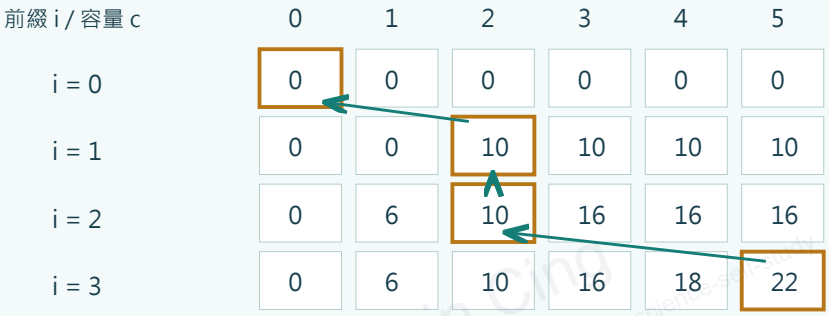
每格只讀前一列；依 $i = 1, 2, 3$ 填列，同列容量順序可任意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一數例：完整狀態表

完整表與回溯：W = 5

物品 (w,v)：1=(2,10), 2=(1,6), 3=(3,12)



箭頭 = 回溯：取3 → 不取2 → 取1；總重5，總值22

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

看懂表格中的三次比較

完整表給出 $D[3,5]=22$。

物品 1：$D[1,1]=0$（太重）；$D[1,2]=\max(0,10+0)=10$。 物品 2：$D[2,3]=\max(D[1,3],6+D[1,2])=\max(10,16)=16$。 物品 3：$D[3,3]=\max(16,12+D[2,0])=16$，能裝不代表必須取；$D[3,4]=\max(16,12+6)=18$；$D[3,5]=\max(16,12+10)=22$。 每個「取」候選皆是當前物品價值加上可行的前綴最優值，不能加同列任意最大值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

回溯把最優值變成物品集合

從 $(i,c)=(n,W)$ 開始；若 $D[i,c]=D[i-1,c]$，選擇不取並令 $i\leftarrow i-1$。否則取 i，令 $c\leftarrow c-w_i$、$i\leftarrow i-1$。相等時固定不取，輸出一組最優解。 本例 $(3,5):22>16$，取 3，至 $(2,2)$；$10=10$，不取 2，至 $(1,2)$；$10>0$，取 1，至 $(0,0)$。 輸出 $\{1,3\}$：重量 $2+3=5$，價值 $10+12=22$。每步 i 減一，回溯 $O(n)$；不要把容量遞減當作唯一停止條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可直接實作的二維版本

```
D[0..n][0..W] = 0
for i = 1..n:
    for c = 0..W:
        D[i,c] = D[i-1,c]
        if c >= w[i]:
            D[i,c] = max(D[i,c], v[i] + D[i-1,c-w[i]])
```

表格有 $(n+1)(W+1)$ 格，每格常數次運算。時間 $O(n(W+1))$，完整表空間 $O(n(W+1))$；通常在 $W\geq 1$ 時寫作 $O(nW)$。保留表格即可依前頁程序重建。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

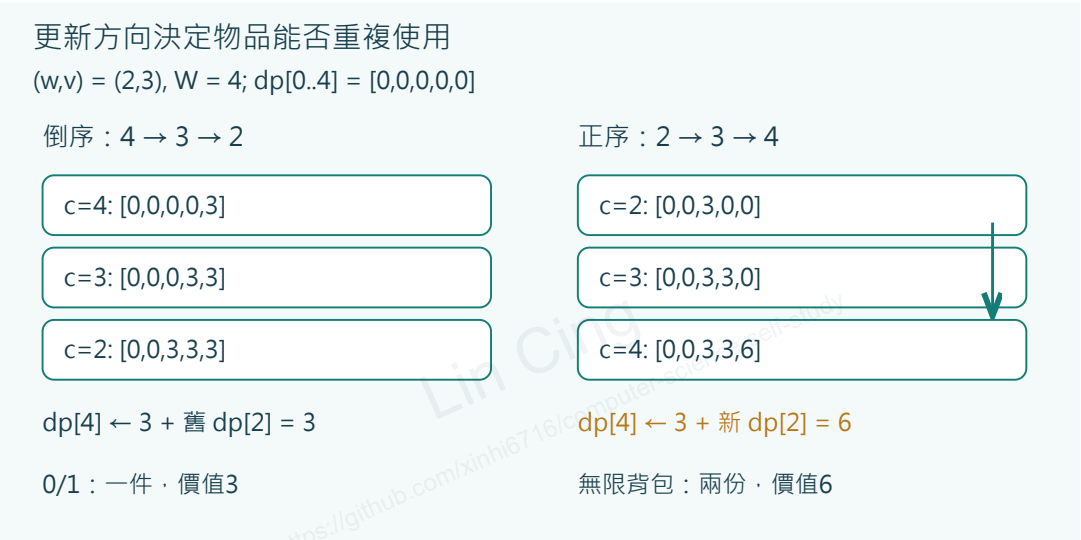
壓成一維：倒序維持前一列

```
dp[0..W] = 0
for i = 1..n:
    for c = W down to w[i]:
        dp[c] = max(dp[c], v[i] + dp[c-w[i]])
```

處理 i 時，尚未更新的容量存 $D[i-1,\cdot]$。倒序且 $w_i > 0$，使 $c - w_i < c$ 尚未更新；右側讀到的仍是前一列。更新後 $dp[c] = D[i,c]$。 時間仍 $O(nW)$，值的空間降至 $O(W)$。被覆寫的歷史不足以直接做二維回溯；需要解集合時，可保留二維表或另設帶版本的決策紀錄。

正序反例：一件物品被拿了兩次

只有一件物品 $(w,v) = (2,3)$，容量 $W = 4$。0/1 最優值應為 3。



正序在 $c = 2$ 寫入 3，之後 $c = 4$ 讀到剛寫的 $dp[2] = 3$，得到 6，等於使用兩份同一物品。更新方向是在選擇依賴哪個版本的狀態。

無限背包：同列依賴正好需要正序

若每種物品可取任意非負整數份，定義 $U[i,c]$ 為前 i 種、重量至多 c 的最大價值。可放時

$$U[i,c] = \max\{U[i-1,c],\ v_i + U[i,c-w_i]\}.$$

取走一份後仍能取第 i 種，所以第二項是同列。以容量正序更新一維陣列即可；因 $w_i > 0$，同列依賴的容量嚴格較小。 例 $(2,3), W=4$：可取兩份，答案 6；此時前頁的正序結果才符合模型。本節排除零重物品；零重正價值且無限供應會使目標無上界，不能套用此有限遞迴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

恰好裝滿：不可行不是零分

令 $E[i,c]$ 為前 i 件、總重**恰為** c 的最大價值。遞迴形式與 0/1 相同，但

$$E[0,0] = 0, \quad E[0,c] = -\infty\ (c > 0).$$

不可行候選保持 $-\infty$；實作先檢查可達再相加，避免整數哨兵溢位。正重量使 $E[i,0] = 0$。 例單件 $(2,-3), W=2$：至多容量答案為 0（空集合）；恰好容量答案為 -3。若 $W=1$，恰好容量不可行。全部初始化為 0 會把不存在的重量誤作可行解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

偽多項式取決於數值大小

$O(nW)$ 對容量數值 W 是多項式，對二進位輸入長度卻未必。記錄容量約需 $\log_2(W+1)$ 位元；若 $W = 2^b$，掃描容量要 2^b 次。 例如同樣 20 件物品，$W = 10^3$ 約有兩萬格更新，$W = 10^9$ 約有兩百億格。整數重量可索引不等於容量索引一定實用。 此分析把每次數值加法與比較視為常數成本；若價值整數位數很大，還需計入算術位元成本。0/1 背包的決定版是 NP-complete；最佳化版稱 NP-hard。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分數背包的貪心為何不同

若改成 $0 \le x_i \le 1$，每件可切分，對正價值物品按 v_i/w_i 由大到小取；負價值可略過。若較低密度被選而較高密度尚未用完，交換相同重量 δ 會使價值不減；反覆交換可得到密度貪心解。 用相同三件物品、容量改為 $W = 4$：密度依序是物品 2 的 6、1 的 5、3 的 4。 分數解取 2、1、$1/3$ 件 3，價值 $6 + 10 + 4 = 20$。0/1 密度貪心取 2、1 得 16；0/1 最優取 2、3 得 18。交換重量若不能切分就未必可行。排序實作為 $O(n \log n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分枝限界的上界不是 DP 格值

對最大化問題，分枝限界法（branch and bound）在取／不取的搜尋樹中，保留已找到的可行解價值 L。令已取物品價值為 V、剩餘容量為 C，只用尚未決定物品的分數背包最優值為 $F(C)$。節點的總上界為 $B = V + F(C)$。 因放寬限制只會增加可行集合，節點任何 0/1 完成解價值都不超過 B；若 $B \le L$，可剪去該節點（只求一組最優解）。 上一例根節點 $B = 20$，整數最優值為 18：上界不一定可達。DP 的 $D[i, c]$ 則是所定義子問題的**確切**最優值；兩者不可混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

子序列保留次序，允許跳過字元

子序列的索引必須嚴格遞增；子字串則必須連續。

固定 $X =$ `ABCBDAB`、$Y =$ `BDCABA`。`BCBA` 在 X 使用索引 $(2, 3, 4, 6)$，在 Y 使用 $(1, 3, 5, 6)$。它的長度為 4；`BCA` 雖也是共同子序列，卻不是最長。

目標先求最長共同子序列（LCS）的**長度**，再重建其中一個解。

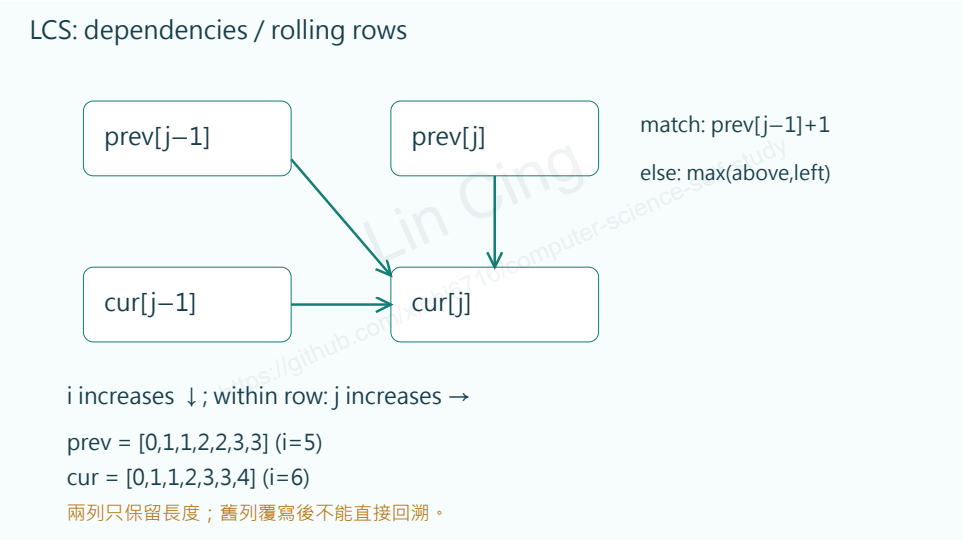
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態必須同時記住兩個前綴

令 $C[i,j]$ 為 $X[1..i]$ 與 $Y[1..j]$ 的 LCS 長度。答案是 $C[7,6]$。

$C[0,j] = C[i,0] = 0$：任一前綴為空，只能取空子序列。

依賴只往較小的 $i+j$，形成 DAG。按照 $i = 1..m$、每列 $j = 1..n$ 填表時，所需的左、上、左上格皆已完成。



相同末字元：先證明可達下界

若 $x_i = y_j = a$，取較短前綴的一個 LCS，再附加 a。索引合法，故

$$C[i,j] \geq C[i-1,j-1] + 1.$$

這證明「候選解做得到」。例如 $(6,6)$ 的末字元皆為 A，$C[5,5] = 3$，故有長度 4 的共同子序列。單有下界尚不能斷言它最優。

相同末字元：再證明沒有更長的解

取任一 LCS Z，長度 k。其末字元必為 a；若不是，兩邊最後位置都未用，可再附加 a，與最長矛盾。

刪掉 Z 最後的 a，其餘字元在兩邊都可嵌入前 $i-1, j-1$ 個位置。因此 $k-1 \le C[i-1, j-1]$，得到上界。結合前頁：

$$C[i, j] = C[i-1, j-1] + 1.$$

這保證存在使用兩個末位置的最佳嵌入；不代表每一種最佳匹配的索引都必須使用它們。例如 A 對 AA 可匹配第一個或第二個 A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不同末字元：兩個方向的界

若 $x_i \neq y_j$，共同子序列的一組嵌入不可能同時使用兩邊最後位置：若兩者都使用，必須是同一個末字元。

所以任一最佳解至少略過 x_i 或 y_j，其長度至多為 $\max(C[i-1, j], C[i, j-1])$。這是上界。

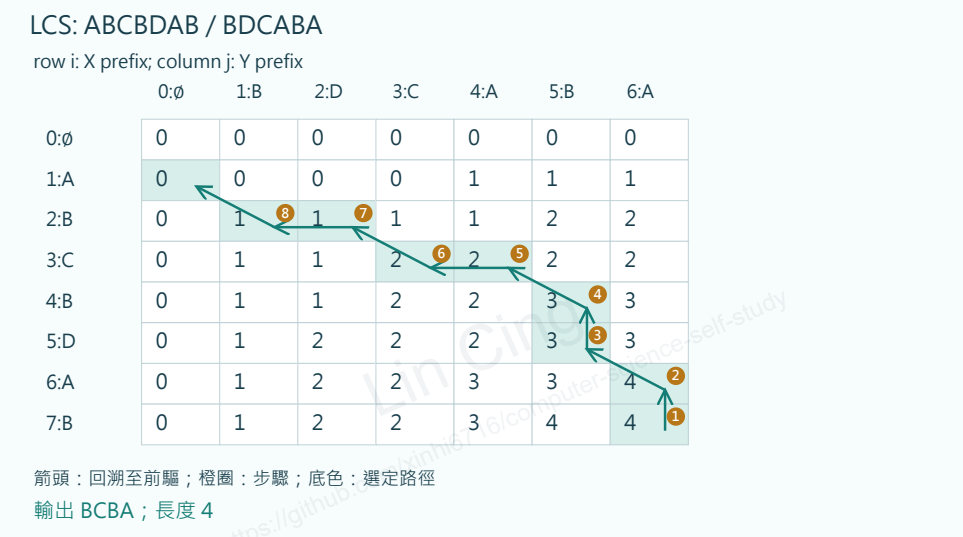
反向看，兩個較短前綴的共同子序列，在完整前綴中仍合法；取較長者即可達到該最大值。故

$$C[i, j] = \max\{C[i-1, j], C[i, j-1]\}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整值表：數值代表長度

相同取左上加 1，不同取上、左最大值。零邊界不可省略。



	0:ø	1:B	2:D	3:C	4:A	5:B	6:A
0:ø	0	0	0	0	0	0	0
1:A	0	0	0	0	1	1	1
2:B	0	1	1	1	1	2	2
3:C	0	1	1	2	2	2	2
4:B	0	1	1	2	2	3	3
5:D	0	1	2	2	2	3	3
6:A	0	1	2	2	3	3	4
7:B	0	1	2	2	3	4	4

例如 $C[3,3] = C[2,2] + 1 = 2$；$C[5,5] = \max(3,2) = 3$；$C[7,6] = \max(4,4) = 4$。編號箭頭是下一頁逐步使用的回溯路線。

逐步回溯：反轉 ABCB 得 BCBA

不同字元且上、左同值時，本例固定先往上；相同則往左上並記錄字元。

步	目前格 → 前驅格	理由／記錄
1	$(7,6) \to (6,6)$	上、左皆 4，先上
2	$(6,6) \to (5,5)$	A=A，記 A
3	$(5,5) \to (4,5)$	上 3 > 左 2
4	$(4,5) \to (3,4)$	B=B，記 B
5	$(3,4) \to (3,3)$	左 2 > 上 1
6	$(3,3) \to (2,2)$	C=C，記 C
7	$(2,2) \to (2,1)$	左 1 > 上 0
8	$(2,1) \to (1,0)$	B=B，記 B；停止

ties 決定選哪一個最佳解

倒序記錄 ABCB，反轉得到 BCBA。

BCBA、BCAB、BDAB 都是長度 4 的 LCS。不同 tie 規則可能回傳不同字串，長度不變。

在 $(7, 6)$ 往左到 $(7, 5)$ 也是合法最佳選擇；往上只是本例的確定性規則。

只存一個前驅只能重建一個解。若要列出所有不同 LCS，必須探索所有合法最佳分支並去重；分支路徑與不同字串的數量也不相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

填表與重建有不同成本

共有 $(m + 1)(n + 1)$ 個狀態，每個內部格只做常數次比較：填完整表時間 $\Theta(mn)$、空間 $\Theta(mn)$。

從 (m, n) 回溯，每一步至少減少一個索引，時間 $O(m + n)$。可存 choice 表，也可保留完整值表，回溯時重新比較鄰格；兩者都能重建一個解。

暴力枚舉一邊的索引子集有 2^m 種候選；許多不同遞迴路徑會重複要求同一對前綴。DP 合併的就是這些重複狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩列滾動：保留長度，丟掉歷史

讓較短的字串放在欄方向。prev 是上一列，cur 是當前列；每列由左至右更新。

```
prev[0..n] = 0
cur[0..n] = 0
for i = 1..m:
    cur[0] = 0
    for j = 1..n:
        if X[i] == Y[j]: cur[j] = prev[j-1] + 1
        else: cur[j] = max(prev[j], cur[j-1])
    swap(prev, cur)
return prev[n]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩列滾動與解的重建

時間仍為 $\Theta(mn)$，額外空間 $O(\min(m, n))$。此程式只回傳長度。

每次更新讀取上一列與當前列的左鄰格。交換兩列後，舊歷史可以覆寫，因為後續長度計算不再需要它。

過去列被覆寫後，不能直接照完整表路線回溯。若要輸出序列，保留完整值／choice 表，或另外使用分治重建等演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

全域對齊：每個字元都必須交代

給定 $X = \texttt{ACGTA}$、$Y = \texttt{AGCAC}$。可插入缺口 `-`，使兩列等長；不得有 `-/-` 欄。刪去缺口後須恢復原序列。

一欄的類型	分數
相同字元	+3
不同字元	−2
一個字元對一個缺口	−3

目標是所有欄分數總和最大，開頭與結尾的缺口也照常扣分。這是線性缺口成本的全域對齊；LCS 長度不是本問題的分數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對齊狀態與完整邊界

$F[i, j]$ 是前綴 $X[1..i]$ 與 $Y[1..j]$ 的最佳全域對齊分數。

$$F[0, 0] = 0, \qquad F[i, 0] = -3i, \qquad F[0, j] = -3j.$$

只有一邊有字元時，每個字元都必須對缺口，因此邊界為 `0, -3, -6, -9, -12, -15`。

不能把邊界全設 0；那會把未對齊的前綴免費略過，改變問題。也不能加上「與 0 取最大」而仍稱為同一個全域對齊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後一欄只有三種可能

令 $s(a,b)=3$（相同）或 -2（不同）。當 $i,j>0$：

$$F[i,j]=\max\begin{cases}F[i-1,j-1]+s(x_i,y_j) & \text{字元對字元}\\ F[i-1,j]-3 & x_i \text{ 對缺口}\\ F[i,j-1]-3 & \text{缺口對 } y_j.\end{cases}$$

任一合法對齊的最後一欄必屬其中一類；刪去後得到相應前綴對齊，故不會超過右側最大值。反過來，任一前驅最佳對齊附加該欄仍合法，能達到該候選值。兩方向共同證明遞推式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對齊完整數例：最佳分數為 1

Global alignment: ACGTA / AGCAC

row i: X prefix; column j: Y prefix

	0:ø	1:A	2:G	3:C	4:A	5:C
0:ø	0	-3	-6	-9	-12	-15
1:A	-3	3	0	-3	-6	-9
2:C	-6	0	1	3	0	-3
3:G	-9	-3	3	0	1	-2
4:T	-12	-6	0	1	-2	-1
5:A	-15	-9	-3	-2	4	1

箭頭：回溯至前驅；橙圈：步驟；底色：選定路徑

輸出 ACGTA- / A-GCAC；總分 1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

$F[2,3]=\max(0+3,-3-3,1-3)=3$。

重建對齊：每一個欄都有代價

從 $(5,5)$ 回溯：

目前格 → 前驅格	附加欄	該步加分
$(5,5) \to (5,4)$	`-/C`	−3
$(5,4) \to (4,3)$	`A/A`	+3
$(4,3) \to (3,2)$	`T/C`	−2
$(3,2) \to (2,1)$	`G/G`	+3
$(2,1) \to (1,1)$	`C/-`	−3
$(1,1) \to (0,0)$	`A/A`	+3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正向讀出重建的對齊

反轉欄順序：

```
ACGTA-
A-GCAC
```

總分 $3-3+3-2+3-3=1$。兩個原序列都完整保留。

兩個缺口欄分別交代 X 的 C 與 Y 最後的 C。T/C 不匹配貢獻 -2，三個匹配欄貢獻 $3+3+3$。

$F[5,5]=\max(-2-2,-1-3,4-3)=1$，故最後一欄選 `-/C`。即使最終分數為正，中間格仍可為負；全域對齊不把它們重設為 0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相似的表格，需要不同的語義

問題	狀態值	相同字元	不同字元	邊界
LCS	最長長度	左上 +1	上、左最大	0
全域對齊	最大總分	三候選仍都比較	三候選仍都比較	累計缺口成本

LCS 可以略過字元而不計費；對齊必須把所有字元放到某一欄。對齊即使字元相同，也依既定三分支規則比較候選，不能直接套用 LCS 的 if/else。

兩者都能用前綴 DAG 填表；完整重建則需要足夠的前驅資訊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

區間 DP：最後一次決策

矩陣鏈最後一次相乘決定切點 k；BST 的根 r 決定左右鍵區間。兩者都保留原本順序，不能任意重排。

狀態 $[i, j]$ 記錄連續區間；依長度遞增填表，因為每個決策只依賴更短區間。值表回答「多好」，決策表回答「如何做」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

矩陣鏈：維度決定合併成本

給定 $p = [5, 10, 3, 12, 5]$，矩陣 A_i 的維度是 $p_{i-1} \times p_i$。

矩陣	A_1	A_2	A_3	A_4
維度	5×10	10×3	3×12	12×5

本節用傳統乘法計純量乘法數：$(a \times b)(b \times c)$ 要 abc 次，結果 $a \times c$。只改括號，不改矩陣順序；DP 規劃成本不包含實際相乘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

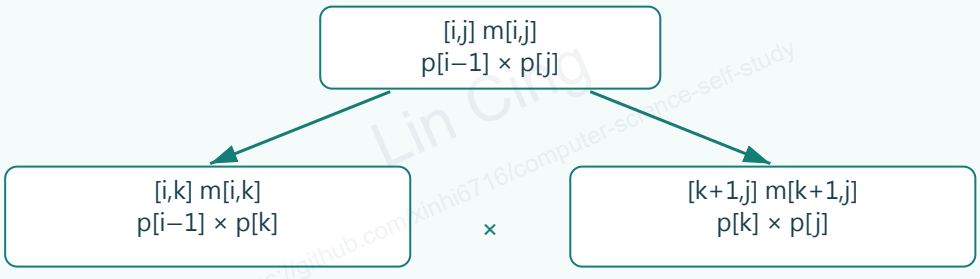
矩陣鏈遞迴與狀態充分性

$m[i, j]$ 是計算 $A_i \cdots A_j$ 的最少乘法數，$m[i, i] = 0$。

$$m[i, j] = \min_{i \le k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \}.$$

任何括號化的區間結果都為 $p_{i-1} \times p_j$，所以外部只需知道端點與最小成本，不需記內部括號。固定切點後兩個子區間的維度都固定。

固定切點：兩側輸出維度不隨括號改變



merge = p[i−1] · p[k] · p[j]

箭頭：父狀態依賴兩個更短區間；替換子解只降低內部成本

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

切開、替換、再合併的證明

任一完整括號化的最後乘法都有某個 $i \le k < j$。若其中左子括號化非最優，換成更便宜的左解，輸出維度不變，右成本與合併成本也不變，總成本下降，矛盾；右側同理。

反方向，每個 k 的兩個可行子解都能合法相乘，因此每個候選都是可行完整解。涵蓋最優解且沒有加入不可行解，取最小值就是最優。以區間長度歸納，已算子表正確即可推出當前格正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

依長度填表：所有短區間候選

所有對角線 $m[i,i]=0$；表列每個切點的「左＋右＋合併」。

長度	區間	候選與計算	m；s
2	$[1,2]$	$k=1: 0+0+5\cdot 10\cdot 3=150$	150；1
2	$[2,3]$	$k=2: 0+0+10\cdot 3\cdot 12=360$	360；2
2	$[3,4]$	$k=3: 0+0+3\cdot 12\cdot 5=180$	180；3
3	$[1,3]$	$k=1: 0+360+600=960$；$k=2: 150+0+180=330$	330；2
3	$[2,4]$	$k=2: 0+180+150=330$；$k=3: 360+0+600=960$	330；2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整鏈：三個切點都必須比較

$[1,4]$ 的切點	左成本	右成本	最後相乘	候選總成本
$k=1$	0	330	$5\cdot10\cdot5=250$	580
$k=2$	150	180	$5\cdot3\cdot5=75$	**405**
$k=3$	330	0	$5\cdot12\cdot5=300$	630

最小合併成本本身不是一般性的貪婪規則，必須加上兩邊的最佳成本。此例巧合由 $k=2$ 同時得到最便宜合併與最便宜總成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整鏈：三個切點都必須比較 （續）

矩陣鏈完整表：m[i,j] / s[i,j]

	j=1	j=2	j=3	j=4
i=1	0 / –	150 / 1	330 / 2	405 / 2
i=2		0 / –	360 / 2	330 / 2
i=3			0 / –	180 / 3
i=4				0 / –

填表順序：L=1 → 2 → 3 → 4；每格只讀更短區間

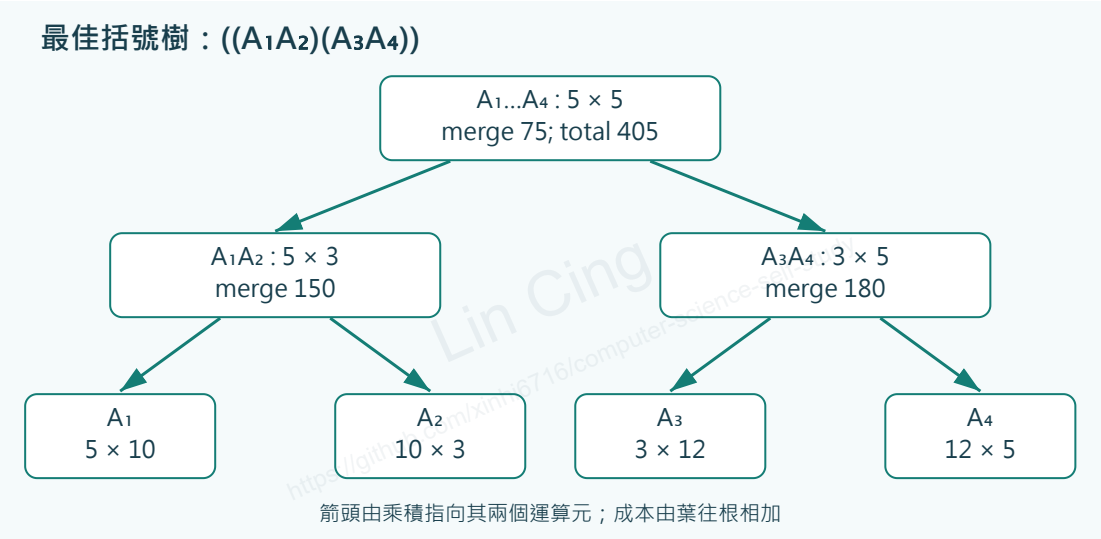
L=4: 580, 405, 630 → min = 405

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由切點表還原括號樹

$s[1,4]=2$，接著 $s[1,2]=1$、$s[3,4]=3$，得 $((A_1A_2)(A_3A_4))$。

先算 A_1A_2：5×3，成本150；算 A_3A_4：3×5，成本180；最後得到 5×5，成本75，總計405。



遞迴：若 $i=j$ 輸出 A_i；否則輸出括號內的 `build(i,s[i,j])` 與 `build(s[i,j]+1,j)`。每個葉與合併節點各訪問一次，重建 $\Theta(n)$。

矩陣鏈演算法與精確工作量

```
for i=1..n: m[i,i]=0
for L=2..n:
  for i=1..n-L+1:
    j=i+L-1; m[i,j]=infinity
    for k=i..j-1:
      t=m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
      if t<m[i,j]: m[i,j]=t; s[i,j]=k
```

長度 L 有 $n-L+1$ 格、各有 $L-1$ 個候選，所以候選數

$$\sum_{L=2}^{n}(n-L+1)(L-1)=\binom{n+1}{3}=\frac{n^3-n}{6}.$$

故時間 $\Theta(n^3)$；值表與切點表各 $\Theta(n^2)$ 空間。這是規劃的複雜度，不是實際矩陣乘法耗時。

OBST：成功與失敗都要建模

有序真鍵 $k_1 < \cdots < k_n$，成功查詢權重 $p_1, \ldots, p_n$。外部葉 $d_0, \ldots, d_n$ 代表查無資料的間隙，權重 $q_0, \ldots, q_n$。

d_0 表示小於 k_1；d_i 表示 $k_i < x < k_{i+1}$；d_n 表示大於 k_n。一棵含 n 個真鍵的擴充 BST 有 $n+1$ 個外部葉。

機率模型有 $\sum p + \sum q = 1$；非負次數權重也能直接使用，總成本除以全域總權重才得到每次查詢期望。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩種成本：差一個固定常數

根深度為0。$C(T)$ 只數實際比較的真鍵；$E(T)$ 額外數到達外部葉的一次終止檢查。

$$C(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j \operatorname{depth}(d_j).$$

$$E(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j(\operatorname{depth}(d_j) + 1).$$

所以 $E(T) - C(T) = \sum_j q_j$，與樹形無關，最優樹集合相同。空段只有 d_{i-1}：$C[i, i-1] = 0$，$E[i, i-1] = q_{i-1}$。初始化與報出的成本必須用同一慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OBST 區間：保留全域機率質量

區間 $[i, j]$ 包含鍵 $k_i, \ldots, k_j$ 與外部葉 $d_{i-1}, \ldots, d_j$。

$$w[i, j] = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t.$$

子問題仍用原始全域權重，不除以區間質量。因為子樹降低一層時，每個屬於此子樹的查詢都多一次檢查，增加的是全域質量 w。

若改存條件期望，合併時必須重新乘回各子樹質量，不能把不同分母的條件期望直接相加。空段的 $w[i, i-1] = q_{i-1}$ 也不能漏掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OBST 遞迴與替換證明

$$E[i, j] = w[i, j] + \min_{i \le r \le j} \{E[i, r-1] + E[r+1, j]\}.$$

C 也滿足相同形狀的遞迴，但空段成本為0。根 r 的左右鍵範圍由中序順序唯一決定；合併增加 $w[i, j]$。

任一最優樹都必有某個根。若其中子樹非最優，以同一鍵／間隙集合的較便宜樹替換，仍為 BST；子樹全域質量不變，所以移至相同深度所增成本不變，得到更小總成本而矛盾。反向，每個候選根搭配兩棵可行子樹都形成可行 BST，因此枚舉所有根後取最小即為最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三鍵例 B：初始化與單鍵區間

固定 $p = (.20, .30, .10)$、$q = (.10, .05, .15, .10)$，總質量1。

空段 $E[1,0] = .10$、$E[2,1] = .05$、$E[3,2] = .15$、$E[4,3] = .10$。

區間	w	唯一候選 $E_L + E_R + w$	E	C
$[1,1]$	.35	$.10 + .05 + .35$	.50	.35
$[2,2]$	.50	$.05 + .15 + .50$	.70	.50
$[3,3]$	.35	$.15 + .10 + .35$	.60	.35

單鍵區間的 C 已含其左右間隙的失敗比較，故不是單純 p_i。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

例 B：兩鍵與三鍵所有根候選

區間；質量	候選根與完整計算	最小 E；C
$[1,2]$；.80	$r=1: .10 + .70 + .80 = 1.60$；$r=2: .50 + .15 + .80 = 1.45$	1.45；1.15
$[2,3]$；.70	$r=2: .05 + .60 + .70 = 1.35$；$r=3: .70 + .10 + .70 = 1.50$	1.35；1.05
$[1,3]$；1.00	$r=1: .10 + 1.35 + 1 = 2.45$；$r=2: .50 + .60 + 1 = 2.10$；$r=3: 1.45 + .10 + 1 = 2.55$	**2.10；1.70**

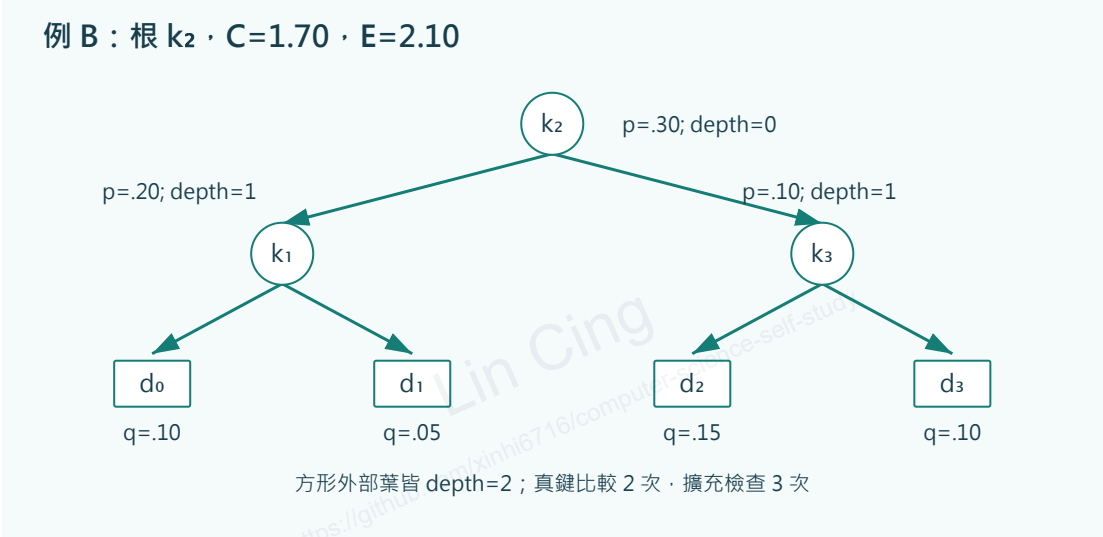
這三個區間的最佳根皆為 $r = 2$。

各區間 $C = E - \sum_{t=i-1}^{j} q_t$；扣除的是該區間的間隙總質量。全表共10個根候選，不能只算最後3個。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

例 B：重建後直接驗算期望

根為 k_2，左子 k_1，右子 k_3。真鍵深度分別1、0、1；四個外部葉深度皆2。



例 B：逐項核對查詢成本

真鍵比較次數依序為2、1、2，成功查詢的加權成本為 $.20 \cdot 2 + .30 \cdot 1 + .10 \cdot 2 = .90$。

四個失敗間隙的總機率為 $.10 + .05 + .15 + .10 = .40$。每次失敗查詢比較2個真鍵；若將外部葉的終止檢查也計入，則成本為3。

$C = .90 + (.10 + .05 + .15 + .10) \cdot 2 = 1.70$；$E = .90 + .40 \cdot 3 = 2.10$。兩值差.40；這是每次全域查詢的期望，不是「成功條件下」期望。

OBST 填表、重建與精確成本

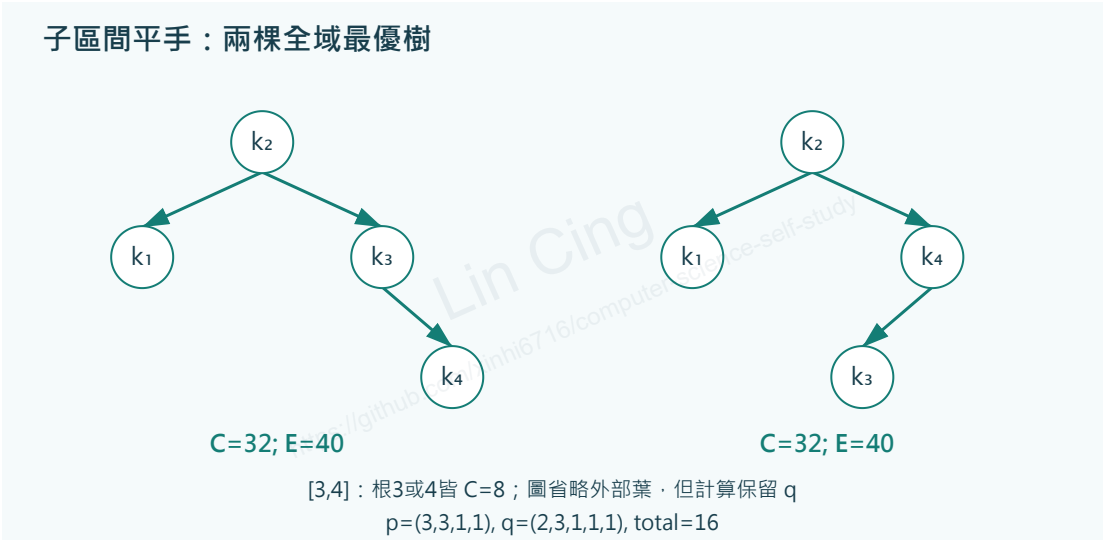
```
for i=1..n+1: E[i,i-1]=w[i,i-1]=q[i-1]
for L=1..n:
  for i=1..n-L+1:
    j=i+L-1; w[i,j]=w[i,j-1]+p[j]+q[j]
    E[i,j]=infinity
    for r=i..j:
      t=E[i,r-1]+E[r+1,j]+w[i,j]
      if t<E[i,j]: E[i,j]=t; root[i,j]=r
```

候選數 $\sum_{L=1}^{n}(n-L+1)L=\binom{n+2}{3}=n(n+1)(n+2)/6$，因此基本演算法 $\Theta(n^3)$ 時間、$\Theta(n^2)$ 空間。`build(i,j)` 若空則回傳 d_{i-1}，否則以 `root[i,j]` 接左右子樹；輸出 $2n+1$ 節點，$\Theta(n)$ 時間。

整數例 C：相同最小成本可以有兩棵樹

取次數權重 $p=(3,3,1,1)$、$q=(2,3,1,1,1)$，全域總權重16。

區間 $[3,4]$ 的 $w=5$；C 候選根3為 $0+3+5=8$，根4為 $3+0+5=8$，兩者皆最優。全域根2的候選為 $C[1,1]+C[3,4]+16=8+8+16=32$；其他根1、3、4的候選分別35、38、41。



全域 $C=32$、$E=40$；除以16得期望2、2.5。只存第一個最小根可重建一棵；要求所有最優樹時須存所有平手根，輸出數量可能指數成長。

僅成功查詢與完整模型如何銜接

模型	權重與邊界	最佳化目標
資料結構課第10章成功版	只有 p；空段0；$W = \sum p$	成功查詢加權真鍵比較數
本節完整模型	p, q；C 空段0，E 空段 q；$w = \sum p + \sum q$	成功與失敗的全域加權成本

現行資料結構教材也另外列出完整 p/q 模型與 $E - C$ 差；不能把「成功版」當成它唯一的慣例。原始資料結構筆記使用真鍵成本 C；CLRS §15.5 使用擴充成本 E，本節明確連接兩者。

將所有 q 設0才退化為僅成功查詢的模型；直接刪去非零 q 會改變最佳化問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

區間 DP 的檢查清單

矩陣鏈：端點決定輸出維度；切點必須 $i \leq k < j$；最小的是完整候選總成本；重建沿記錄的切點。

OBST：鍵順序固定；間隙隨子區間保留；空段成本與 C/E 慣例一致；保留全域權重；所有候選根都比較；平手策略只影響選哪棵最優樹。

矩陣鏈與 OBST 的「同為區間 DP」不代表可互換合併成本或邊界。此處分析基本立方演算法；更快的 OBST 最佳化須另外證明根單調性，不能套到任意區間 DP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 15, Dynamic Programming. MIT 6.046J, Fall 2004, Lecture 12, Dynamic Programming. 講者 Charles E. Leiserson。 MIT 6.046J, Fall 2005, Lecture 15, Dynamic Programming. 講者 Charles E. Leiserson；投影片署名 Erik D. Demaine、Charles E. Leiserson。 原始掃描筆記《演算法_wjungle》動態規劃相關頁；《資料結構》實體第 166–170 頁用於比較最佳搜尋樹模型。原始筆記涵蓋演算法與資料結構，並非單一本教科書全文。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study